

Código para estudiar

Dynamic Augmented Pooled Testing — archivos clave

2026-05-25

Cómo usar este PDF

Este PDF junta el **código clave** del proyecto en un solo archivo, para que lo puedas subir a una IA tutora (que solo acepta PDFs) y estudiarlo. Léelo **junto al PDF estado_proyecto_2026-05.pdf**, que es el mapa del proyecto.

Orden sugerido de lectura (de lo simple a lo complejo):

1. `core.py` — las piezas básicas.
2. `solver.py` — la respuesta *óptima* exacta (la referencia).
3. `bayesian.py` — cómo se actualizan las probabilidades.
4. `greedy.py` — la estrategia rápida (avariciosa).
5. `vw_demo.py` y `vw_restrict.py` — la idea de super-nodos.
6. `rl_examples.py` — RL básico (coincide con el óptimo).
7. `rl_env.py` y `rl_train.py` — RL con PPO (lo nuevo).

Cómo estudiarlo con la IA (modo activo, no pasivo):

- Pídele: *“hazme preguntas sobre este código, NO me lo expliques”*.
- Por cada archivo: intenta tú decir qué hace cada función *antes* de leer la respuesta de la IA, y que ella te corrija.
- Pídele un ejemplo chico ($n=3$) y que te haga predecir la salida.

Nota: hay más archivos en el repo (por ejemplo `hybrid_solver.py`, `independence_gap.py`, `semi_utility.py`) que no se incluyen aquí para no saturar; están descritos en el PDF de estado.

Contents

1	<code>core.py</code>	2
2	<code>solver.py</code>	4
3	<code>bayesian.py</code>	6
4	<code>greedy.py</code>	14
5	<code>vw_demo.py</code>	20
6	<code>vw_restrict.py</code>	24
7	<code>rl_examples.py</code>	29
8	<code>rl_env.py</code>	34
9	<code>rl_train.py</code>	39

1 core.py

Piezas básicas: representar conjuntos como bits, generar todos los pools posibles, y calcular el resultado de un test (cuántos infectados hay en un grupo).

```
1  """
2  Bitmask helpers and core primitives for DAPTS.
3
4  Individuals are indexed 0..n-1. A "mask" is a Python int whose bit i
5  is set iff individual i is in the set (pool, infection profile, etc.).
6  """
7
8  from itertools import combinations
9
10
11 def mask_from_indices(indices):
12     """Bitmask with the given bit positions set. e.g. [0,2] -> 0b101."""
13     m = 0
14     for i in indices:
15         m |= 1 << i
16     return m
17
18
19 def indices_from_mask(mask, n=None):
20     """Sorted list of set-bit positions. If n given, ignore bits >= n."""
21     if n is not None:
22         mask &= (1 << n) - 1
23     out = []
24     while mask:
25         lsb = mask & -mask
26         out.append(lsb.bit_length() - 1)
27         mask ^= lsb
28     return out
29
30
31 def popcount(mask):
32     """Number of set bits."""
33     return mask.bit_count()
34
35
36 def mask_str(mask, n=None):
37     """Human-readable string for a bitmask. e.g. 0b101 -> '{0,2}'."""
38     idxs = indices_from_mask(mask, n)
39     return "{" + ",".join(str(i) for i in idxs) + "}" if idxs else "{}"
40
41
42 def all_pools(n, G, include_empty=True):
43     """Every pool mask t with |t| <= G over population 0..n-1."""
44     pools = []
45     start = 0 if include_empty else 1
46     for size in range(start, min(G, n) + 1):
47         for combo in combinations(range(n), size):
48             pools.append(mask_from_indices(combo))
49     return pools
50
51
52 def compute_active_mask(p, cleared_mask, n, threshold=1e-10):
53     """Bitmask of individuals whose infection status is still uncertain.
54
55     An individual is *inactive* (excluded from future pools) if:
56     - already cleared (bit set in cleared_mask), or
57     - p_i <= threshold (essentially known healthy), or
58     - p_i >= 1 - threshold (essentially confirmed infected).
59
60     Returns (active_mask, confirmed_infected_mask).
61     """
62     active = 0
63     confirmed_infected = 0
64     for i in range(n):
65         if cleared_mask >> i & 1:
66             continue # already cleared
67         if p[i] <= threshold:
```

```

68         continue # known healthy
69     if p[i] >= 1.0 - threshold:
70         confirmed_infected |= 1 << i
71         continue # confirmed infected
72     active |= 1 << i
73     return active, confirmed_infected
74
75
76 def all_pools_from_mask(active_mask, G, include_empty=True):
77     """Generate pool masks using only individuals in active_mask.
78
79     Like all_pools but restricted to the set bits of active_mask.
80     """
81     active_indices = indices_from_mask(active_mask)
82     m = len(active_indices)
83     pools = []
84     start = 0 if include_empty else 1
85     for size in range(start, min(G, m) + 1):
86         for combo in combinations(active_indices, size):
87             pools.append(mask_from_indices(combo))
88     return pools
89
90
91 def test_result(pool_mask, z_mask):
92     """Augmented test result  $r(t, Z) = |t \cap Z|$  (count of infected in pool)."""
93     return popcount(pool_mask & z_mask)

```

2 solver.py

El solver óptimo exacto por programación dinámica (DP). Es la respuesta perfecta contra la que comparamos todo. Solo aguanta n pequeño.

```
1  """
2  Brute-force DP solver for optimal DAPTS on tiny instances (n <= 14).
3
4  State = (step k, remaining_set, cleared_mask)
5  k:      tests already used (0..B)
6  remaining_set: frozenset of z_masks consistent with observations
7  cleared_mask: individuals proven healthy
8  """
9
10 from augmented.core import all_pools, test_result
11 from augmented.strategy import DAPTS
12
13 _MAX_N = 14
14
15
16 def solve_optimal_dapts(p, u, B, G):
17     """Solve for the optimal DAPTS via brute-force DP.
18
19     Returns (optimal_value, optimal_policy).
20     """
21     n = len(p)
22     if n > _MAX_N:
23         raise ValueError(f"Brute-force requires n <= {_MAX_N}, got {n}")
24     if n == 0:
25         return 0.0, DAPTS(B)
26
27     q = [1.0 - pi for pi in p]
28
29     # Pr(Z = z) for every infection profile
30     num_profiles = 1 << n
31     w = [0.0] * num_profiles
32     for z in range(num_profiles):
33         wz = 1.0
34         for i in range(n):
35             wz *= p[i] if (z >> i & 1) else q[i]
36         w[z] = wz
37
38     pools = all_pools(n, G, include_empty=False)
39     memo = {} # (k, remaining, cleared) -> (value, best_pool)
40
41     def _cleared_utility(cleared_mask):
42         total = 0.0
43         m = cleared_mask
44         while m:
45             i = (m & -m).bit_length() - 1
46             total += u[i]
47             m &= m - 1
48         return total
49
50     def dp(k, remaining, cleared_mask):
51         state = (k, remaining, cleared_mask)
52         if state in memo:
53             return memo[state]
54
55         # Terminal: no more tests
56         if k == B:
57             total_mass = sum(w[z] for z in remaining)
58             result = (total_mass * _cleared_utility(cleared_mask), 0)
59             memo[state] = result
60             return result
61
62         total_mass = sum(w[z] for z in remaining)
63         if total_mass == 0.0:
64             memo[state] = (0.0, 0)
65             return memo[state]
66
67         best_value, best_pool = -1.0, 0
```

```

68
69     for pool in pools:
70         # Partition remaining profiles by outcome r
71         buckets = {}
72         for z in remaining:
73             r = test_result(pool, z)
74             buckets.setdefault(r, []).append(z)
75
76         ev = 0.0
77         for r, z_list in buckets.items():
78             new_cleared = cleared_mask | pool if r == 0 else cleared_mask
79             sub_val, _ = dp(k + 1, frozenset(z_list), new_cleared)
80             ev += sub_val
81
82         if ev > best_value:
83             best_value, best_pool = ev, pool
84
85         # Also consider wasting a test (empty pool)
86         waste_val, _ = dp(k + 1, remaining, cleared_mask)
87         if waste_val > best_value:
88             best_value, best_pool = waste_val, 0
89
90         memo[state] = (best_value, best_pool)
91         return memo[state]
92
93 # Solve
94 all_z = frozenset(range(num_profiles))
95 optimal_value, _ = dp(0, all_z, 0)
96
97 # Reconstruct policy from DP argmax decisions
98 policy = DAPTS(B)
99
100 def reconstruct(k, remaining, cleared_mask, history):
101     if k == B:
102         return
103     _, best_pool = memo[(k, remaining, cleared_mask)]
104     policy.set_action(k + 1, history, best_pool)
105
106     buckets = {}
107     for z in remaining:
108         r = test_result(best_pool, z)
109         buckets.setdefault(r, []).append(z)
110
111     for r, z_list in buckets.items():
112         new_cleared = cleared_mask | best_pool if r == 0 else cleared_mask
113         reconstruct(k + 1, frozenset(z_list), new_cleared,
114                     history + ((best_pool, r),))
115
116 reconstruct(0, all_z, 0, ())
117 return optimal_value, policy

```

3 bayesian.py

Cómo actualizar las probabilidades de infección al ver resultados: por un solo test, por la historia completa (capta la info cruzada), y por Gibbs (aproximado, para cuando lo exacto explota).

```
1  """
2  Bayesian posterior update for augmented pooled tests.
3
4  Given prior infection probabilities p = (p_1, ..., p_n) and a test history
5  H_k = ((t_1, r_1), ..., (t_k, r_k)), compute posterior probabilities
6  q'_i = P(individual i is healthy | H_k) for each individual.
7
8  In the idealized augmented model, testing pool t yields r = |t ∩ Z|
9  (the exact count of infected in the pool).
10 """
11
12 from augmented.core import indices_from_mask, test_result, popcount
13
14
15 def _poisson_binomial_pmf(probs):
16     """PMF of the Poisson-Binomial distribution for independent Bernoullis.
17
18     Given probabilities [p_1, ..., p_m], returns a list pmf of length m+1
19     where pmf[k] = P(exactly k successes).
20
21     Uses the DP recurrence:
22         dp[0] = 1
23         dp[k] = dp[k] * (1-p_j) + dp[k-1] * p_j    (for each new p_j)
24     """
25     m = len(probs)
26     dp = [0.0] * (m + 1)
27     dp[0] = 1.0
28     for j, pj in enumerate(probs):
29         # Traverse backwards to avoid overwriting values we still need
30         for k in range(j + 1, 0, -1):
31             dp[k] = dp[k] * (1.0 - pj) + dp[k - 1] * pj
32         dp[0] *= (1.0 - pj)
33     return dp
34
35
36 def bayesian_update_single_test(p, pool_mask, r, n):
37     """Update infection probabilities after one augmented test.
38
39     Parameters
40     -----
41     p : list[float]
42         Prior infection probabilities (length n).
43     pool_mask : int
44         Bitmask of the tested pool t.
45     r : int
46         Observed result r = |t ∩ Z| (count of infected in pool).
47     n : int
48         Population size.
49
50     Returns
51     -----
52     list[float]
53         Posterior infection probabilities p'_i = P(Z_i=1 | r, t).
54
55     Math
56     ----
57     For i NOT in t: test gives no info, so p'_i = p_i.
58
59     For i IN t, by Bayes:
60         p'_i = P(Z_i=1 | r) = P(r | Z_i=1) * p_i / P(r)
61
62     where (letting S = t \ {i}):
63         P(r | Z_i=1) = P(exactly r-1 infected in S)    [Poisson-Binomial]
64         P(r | Z_i=0) = P(exactly r infected in S)      [Poisson-Binomial]
65         P(r)          = P(r|Z_i=1)*p_i + P(r|Z_i=0)*q_i
66     """
67     pool_indices = indices_from_mask(pool_mask, n)
```

```

68
69     if not pool_indices:
70         return list(p)
71
72     posterior = list(p)
73
74     # For each i in pool, compute posterior via Poisson-Binomial on t\{i}
75     for i in pool_indices:
76         # Handle deterministic cases: p_i = 0 or p_i = 1
77         if p[i] <= 0.0:
78             posterior[i] = 0.0
79             continue
80         if p[i] >= 1.0:
81             posterior[i] = 1.0
82             continue
83
84         others = [j for j in pool_indices if j != i]
85         others_p = [max(0.0, min(1.0, p[j])) for j in others]
86
87         # PMF of number of infected among others
88         pmf = _poisson_binomial_pmf(others_p)
89
90         # P(r | Z_i = 1) = P(r-1 infected among others)
91         p_r_given_1 = pmf[r - 1] if r >= 1 else 0.0
92         # P(r | Z_i = 0) = P(r infected among others)
93         p_r_given_0 = pmf[r] if r <= len(others) else 0.0
94
95         # Bayes
96         numerator = p_r_given_1 * p[i]
97         denominator = numerator + p_r_given_0 * (1.0 - p[i])
98
99         if denominator > 0:
100             posterior[i] = numerator / denominator
101         # else: degenerate case, keep prior
102
103     return posterior
104
105
106 def bayesian_update(p, history, n):
107     """Apply Bayesian updates for a full test history (sequential).
108
109     Parameters
110     -----
111     p : list[float]
112         Prior infection probabilities (length n).
113     history : tuple of (pool_mask, result) pairs
114         Test history H_k = ((t_1, r_1), ..., (t_k, r_k)).
115     n : int
116         Population size.
117
118     Returns
119     -----
120     list[float]
121         Posterior infection probabilities after all tests in history.
122     """
123     current_p = list(p)
124     for pool_mask, r in history:
125         current_p = bayesian_update_single_test(current_p, pool_mask, r, n)
126     return current_p
127
128
129 def bayesian_update_by_counting(p, history, n):
130     """Compute posterior P(Z_i=1 | h_k) by counting over all consistent worlds.
131
132     This is the "by counting" approach: enumerate all 2^n infection profiles,
133     keep those consistent with the full test history, and compute posteriors
134     as weighted proportions.
135
136     Parameters
137     -----
138     p : list[float]
139         Prior infection probabilities (length n).

```

```

140 history : tuple of (pool_mask, result) pairs
141     Full test history H_k = ((t_1, r_1), ..., (t_k, r_k)).
142 n : int
143     Population size.
144
145 Returns
146 -----
147 list[float]
148     Posterior infection probabilities P(Z_i=1 | h_k).
149
150 Notes
151 -----
152 Complexity: O(2^n * k) where k = len(history).
153 Feasible for n <= ~20. For independent priors, this gives the same
154 result as sequential Bayesian updates but makes the joint computation
155 explicit and transparent.
156 """
157 if not history:
158     return list(p)
159
160 q = [1.0 - pi for pi in p]
161 num_profiles = 1 << n
162
163 # Accumulate: weighted count of consistent profiles, and per-individual
164 # weighted count of consistent profiles where Z_i = 1
165 total_weight = 0.0
166 infected_weight = [0.0] * n
167
168 for z_mask in range(num_profiles):
169     # Check consistency with ALL tests in history
170     consistent = True
171     for pool_mask, r in history:
172         if test_result(pool_mask, z_mask) != r:
173             consistent = False
174             break
175
176     if not consistent:
177         continue
178
179     # Pr(Z = z_mask) under the independent prior
180     w = 1.0
181     for i in range(n):
182         w *= p[i] if (z_mask >> i & 1) else q[i]
183
184     total_weight += w
185     # Add weight to each infected individual in this profile
186     bits = z_mask
187     while bits:
188         lsb = bits & -bits
189         i = lsb.bit_length() - 1
190         infected_weight[i] += w
191         bits ^= lsb
192
193 # Compute posteriors
194 posterior = list(p) # fallback to prior if degenerate
195 if total_weight > 0:
196     for i in range(n):
197         posterior[i] = infected_weight[i] / total_weight
198
199 return posterior
200
201
202 def gibbs_update(p, history, n, num_iterations=1000, burn_in=200,
203                 window_size=50, tolerance=1e-4, seed=None):
204     """Approximate posterior marginals via Gibbs sampling (MCMC).
205
206     Adapted from Appendix A.2 of "Dynamic Welfare-Maximizing Pooled Testing"
207     for augmented tests where
208     each test returns the exact count r = |t n Z| of infected in the pool.
209
210     The algorithm:
211     1. Preprocessing: deterministic deductions (r=0 → all healthy,

```

```

212     r=|pool| → all infected), with constraint propagation.
213 2. Initialize state vector by sampling from priors.
214 3. For very small active subproblems, fall back to exact counting
215    to avoid mixing pathologies in disconnected feasible regions.
216 4. Gibbs iterations: for each agent in random order, compute the
217    conditional  $P(X_i \mid X_{-i})$  by checking consistency with all
218    test constraints, then sample or force deterministically.
219 5. Add Metropolis-Hastings swap proposals, both within individual
220    test pools and across the full active set, to improve mixing.
221 6. After burn-in, collect samples and estimate marginals as
222    empirical frequencies.
223
224 Parameters
225 -----
226 p : list[float]
227     Prior infection probabilities (length n).
228 history : tuple of (pool_mask, result) pairs
229     Full test history  $H_k = ((t_1, r_1), \dots, (t_k, r_k))$ .
230 n : int
231     Population size.
232 num_iterations : int
233     Maximum number of Gibbs iterations.
234 burn_in : int
235     Number of initial iterations to discard.
236 window_size : int
237     Rolling window size for convergence monitoring.
238 tolerance : float
239     Convergence threshold: stop if max change in marginals < tolerance.
240 seed : int or None
241     Random seed for reproducibility.
242
243 Returns
244 -----
245 list[float]
246     Posterior infection probabilities  $P(Z_{i=1} \mid h_k)$ .
247
248 Notes
249 -----
250 Complexity:  $O(n * |history| * num\_iterations)$ .
251 Scales to  $n \sim 50+$ . For small  $n$ , results approximate those from
252 bayesian_update_by_counting (which is exact but  $O(2^n)$ ); for very
253 small active subproblems, this function uses exact counting directly.
254
255 import random as _random
256
257 if not history:
258     return list(p)
259
260 rng = _random.Random(seed)
261
262 # ---- Step 1: Preprocessing - deterministic deductions ----
263 confirmed_healthy = set()
264 confirmed_infected = set()
265
266 remaining_tests = [(pool_mask, r) for pool_mask, r in history]
267 changed = True
268 while changed:
269     changed = False
270     new_tests = []
271     for pool_mask, r in remaining_tests:
272         # Remove confirmed agents from this test
273         eff_pool = pool_mask
274         eff_r = r
275         for i in confirmed_healthy:
276             if eff_pool >> i & 1:
277                 eff_pool ^= (1 << i)
278         for i in confirmed_infected:
279             if eff_pool >> i & 1:
280                 eff_pool ^= (1 << i)
281                 eff_r -= 1
282
283     pool_size = popcount(eff_pool)

```

```

284
285     if eff_r == 0 and eff_pool != 0:
286         # All remaining in pool are healthy
287         for i in range(n):
288             if eff_pool >> i & 1 and i not in confirmed_healthy:
289                 confirmed_healthy.add(i)
290                 changed = True
291     elif eff_r == pool_size and pool_size > 0:
292         # All remaining in pool are infected
293         for i in range(n):
294             if eff_pool >> i & 1 and i not in confirmed_infected:
295                 confirmed_infected.add(i)
296                 changed = True
297     elif eff_r > 0 and pool_size > 0:
298         new_tests.append((eff_pool, eff_r))
299     remaining_tests = new_tests
300
301     # Build posterior for confirmed agents
302     posterior = list(p)
303     for i in confirmed_healthy:
304         posterior[i] = 0.0
305     for i in confirmed_infected:
306         posterior[i] = 1.0
307
308     # Identify active agents (those in at least one remaining test)
309     active_set = set()
310     for pool_mask, r in remaining_tests:
311         for i in range(n):
312             if pool_mask >> i & 1:
313                 active_set.add(i)
314
315     if not active_set:
316         return posterior
317
318     active_list = sorted(active_set)
319
320     # Exact counting is cheap for tiny active subproblems and avoids
321     # disconnected-state mixing failures in overlapping exact-count tests.
322     if len(active_list) <= 7:
323         return bayesian_update_by_counting(p, history, n)
324
325     # For each active agent, precompute which tests involve them
326     agent_tests = {i: [] for i in active_list}
327     for idx, (pool_mask, r) in enumerate(remaining_tests):
328         for i in active_list:
329             if pool_mask >> i & 1:
330                 agent_tests[i].append(idx)
331
332     # ---- Step 2: Initialize state consistently ----
333     # Try to find a consistent initial state via constraint satisfaction
334     state = {i: 0 for i in active_list}
335
336     # Greedily assign infected agents to satisfy test counts
337     for pool_mask, r in remaining_tests:
338         pool_agents = [j for j in active_list if pool_mask >> j & 1]
339         current_count = sum(state[j] for j in pool_agents)
340         needed = r - current_count
341         if needed > 0:
342             # Infect agents with highest prior probability first
343             healthy_in_pool = [j for j in pool_agents if state[j] == 0]
344             healthy_in_pool.sort(key=lambda j: p[j], reverse=True)
345             for j in healthy_in_pool[:needed]:
346                 state[j] = 1
347
348     # Helper: check if current state satisfies all test constraints
349     def _state_valid():
350         for pm, r_val in remaining_tests:
351             cnt = sum(state[j] for j in active_list if pm >> j & 1)
352             if cnt != r_val:
353                 return False
354         return True
355

```

```

356 # Helper: count infected in a test pool
357 def _count_infected(test_idx):
358     pm = remaining_tests[test_idx][0]
359     return sum(state[j] for j in active_list if pm >> j & 1)
360
361 # ---- Step 3: Gibbs iterations with swap moves ----
362 healthy_counts = {i: 0 for i in active_list}
363 total_samples = 0
364 prev_marginals = None
365
366 for iteration in range(num_iterations):
367     # --- Standard Gibbs sweep ---
368     order = list(active_list)
369     rng.shuffle(order)
370
371     for i in order:
372         infected_ok = True
373         healthy_ok = True
374
375         for test_idx in agent_tests[i]:
376             pool_mask, r = remaining_tests[test_idx]
377             other_infected = 0
378             for j in active_list:
379                 if j != i and (pool_mask >> j & 1) and state[j] == 1:
380                     other_infected += 1
381
382             if other_infected + 1 != r:
383                 infected_ok = False
384             if other_infected != r:
385                 healthy_ok = False
386
387             if infected_ok and healthy_ok:
388                 state[i] = 1 if rng.random() < p[i] else 0
389             elif infected_ok:
390                 state[i] = 1
391             elif healthy_ok:
392                 state[i] = 0
393             # else: neither consistent - keep current state
394
395     # --- Swap moves (Metropolis-Hastings) ---
396     # For each test with  $0 < r < |pool|$ , propose swapping an infected
397     # and a healthy member. This ensures ergodicity for exact-count
398     # constraints where standard Gibbs gets stuck.
399     for test_idx, (pool_mask, r) in enumerate(remaining_tests):
400         infected_in_pool = [j for j in active_list
401                             if (pool_mask >> j & 1) and state[j] == 1]
402         healthy_in_pool = [j for j in active_list
403                            if (pool_mask >> j & 1) and state[j] == 0]
404
405         if not infected_in_pool or not healthy_in_pool:
406             continue
407
408         i_inf = rng.choice(infected_in_pool)
409         i_hlt = rng.choice(healthy_in_pool)
410
411         # Temporarily swap
412         state[i_inf], state[i_hlt] = 0, 1
413
414         # Check all tests involving either agent
415         swap_valid = True
416         for tidx in set(agent_tests[i_inf]) | set(agent_tests[i_hlt]):
417             if _count_infected(tidx) != remaining_tests[tidx][1]:
418                 swap_valid = False
419                 break
420
421         if swap_valid:
422             # Metropolis-Hastings acceptance ratio based on priors
423             p_new = p[i_hlt] * (1.0 - p[i_inf])
424             p_old = p[i_inf] * (1.0 - p[i_hlt])
425             acceptance = min(1.0, p_new / p_old) if p_old > 0 else 1.0
426
427             if rng.random() >= acceptance:

```

```

428         state[i_inf], state[i_hlt] = 1, 0 # reject
429     else:
430         state[i_inf], state[i_hlt] = 1, 0 # revert invalid swap
431
432     # --- Global pairwise block moves ---
433     # These proposals are not restricted to a single test pool, so they
434     # can move across broader feasible regions when pools overlap.
435     n_block_moves = max(len(active_list), 5)
436     for _ in range(n_block_moves):
437         inf_agents = [j for j in active_list if state[j] == 1]
438         hlt_agents = [j for j in active_list if state[j] == 0]
439
440         if not inf_agents or not hlt_agents:
441             break
442
443         i_inf = rng.choice(inf_agents)
444         i_hlt = rng.choice(hlt_agents)
445         state[i_inf], state[i_hlt] = 0, 1
446
447         if _state_valid():
448             p_new = p[i_hlt] * (1.0 - p[i_inf])
449             p_old = p[i_inf] * (1.0 - p[i_hlt])
450             acceptance = min(1.0, p_new / p_old) if p_old > 0 else 1.0
451
452             if rng.random() >= acceptance:
453                 state[i_inf], state[i_hlt] = 1, 0 # reject
454         else:
455             state[i_inf], state[i_hlt] = 1, 0 # revert invalid swap
456
457     # ---- Step 4: Collect samples after burn-in ----
458     if iteration >= burn_in:
459         for i in active_list:
460             if state[i] == 0:
461                 healthy_counts[i] += 1
462             total_samples += 1
463
464     # Convergence check
465     if total_samples > 0 and total_samples % window_size == 0:
466         current_marginals = {i: 1.0 - healthy_counts[i] / total_samples
467                               for i in active_list}
468         if prev_marginals is not None:
469             max_diff = max(abs(current_marginals[i] - prev_marginals[i])
470                             for i in active_list)
471             if max_diff < tolerance:
472                 break
473         prev_marginals = current_marginals
474
475     # Compute posteriors from samples
476     if total_samples > 0:
477         for i in active_list:
478             posterior[i] = 1.0 - healthy_counts[i] / total_samples
479
480     return posterior
481
482
483 def estimate_p_from_history(history, n, prior_p=None, prior_strength=1.0):
484     """Estimate infection probabilities from observed test data.
485
486     Uses a Bayesian approach with a Beta prior. If prior_p is given,
487     uses Beta(prior_strength * p_i, prior_strength * (1 - p_i)) as prior.
488     Otherwise uses uniform Beta(1, 1).
489
490     Parameters
491     -----
492     history : tuple of (pool_mask, result) pairs
493             Test history.
494     n : int
495         Population size.
496     prior_p : list[float] or None
497         Prior guess for infection probabilities.
498     prior_strength : float
499         Strength of the prior (pseudo-count scale).

```

```

500
501 Returns
502 -----
503 list[float]
504     Estimated infection probabilities.
505 """
506 if prior_p is not None:
507     alpha = [prior_strength * pi for pi in prior_p]
508     beta = [prior_strength * (1.0 - pi) for pi in prior_p]
509 else:
510     alpha = [1.0] * n
511     beta = [1.0] * n
512
513 # For each consistent world, accumulate posterior mass
514 if not history:
515     return [a / (a + b) for a, b in zip(alpha, beta)]
516
517 q_prior = [1.0] * n
518 if prior_p:
519     q_prior = [1.0 - pi for pi in prior_p]
520 else:
521     q_prior = [0.5] * n
522
523 p_prior = prior_p if prior_p else [0.5] * n
524
525 # Use counting-based posterior
526 return bayesian_update_by_counting(p_prior, history, n)

```

4 greedy.py

La estrategia rápida: en cada paso elige el pool que se ve mejor ahora. Incluye variantes (miope, lookahead de primer paso, por conteo).

```
1  """
2  Greedy algorithms for augmented pooled testing.
3
4  Three greedy strategies:
5      1. greedy_myopic: at each step, pick pool maximizing  $P(r=0) * \sum u_i$ 
6         with sequential single-test Bayesian updates.
7      2. greedy_lookahead: at each step, pick pool maximizing expected utility
8         over ALL possible outcomes ( $r=0,1,\dots,|t|$ ) with Bayesian updates.
9      3. greedy_myopic_counting: same as myopic but uses full-history
10         Bayesian update by counting over all consistent worlds.
11
12  Key insight: the myopic pool selection is identical for classical and
13  augmented tests (only  $r=0$  matters for immediate utility). The augmented
14  information helps in FUTURE steps through better Bayesian posteriors.
15  """
16
17  from augmented.core import (all_pools, all_pools_from_mask, compute_active_mask,
18                              indices_from_mask, test_result, mask_str)
19  from augmented.bayesian import (bayesian_update_single_test,
20                                  bayesian_update_by_counting,
21                                  gibbs_update,
22                                  _poisson_binomial_pmf)
23
24
25  # -----
26  # Myopic greedy: maximize immediate expected gain
27  # -----
28
29  def _myopic_best_pool(p, u, G, n, cleared_mask, use_filtering=True):
30      """Pick pool maximizing  $P(r=0) * \sum u_i$  for uncleared members.
31
32      Score(t) =  $\prod (1-p_i \text{ for } i \text{ in } t) * \sum (u_i \text{ for } i \text{ in } t \text{ if } i \text{ not cleared})$ 
33
34      If use_filtering=True, only considers pools from active individuals
35      (those not yet cleared and not confirmed infected).
36      """
37      if use_filtering:
38          active_mask, _ = compute_active_mask(p, cleared_mask, n)
39          if active_mask == 0:
40              return 0 # no uncertain individuals left
41          pools = all_pools_from_mask(active_mask, G, include_empty=False)
42      else:
43          pools = all_pools(n, G, include_empty=False)
44
45      best_pool, best_score = 0, 0.0
46
47      for pool in pools:
48          pool_idx = indices_from_mask(pool, n)
49          prob_clear = 1.0
50          for i in pool_idx:
51              prob_clear *= (1.0 - p[i])
52
53          gain = sum(u[i] for i in pool_idx if not (cleared_mask >> i & 1))
54          score = prob_clear * gain
55
56          if score > best_score:
57              best_score = score
58              best_pool = pool
59
60      return best_pool
61
62
63  def greedy_myopic_simulate(p, u, B, G, z_mask, pool_selector=None):
64      """Simulate myopic greedy on a fixed infection profile z_mask.
65
66      At each step: pick best myopic pool, observe augmented result,
67      update beliefs via Bayesian update, repeat.
```

```

68
69 Parameters
70 -----
71 pool_selector : callable or None
72     If provided, called as pool_selector(p, u, G, n, cleared_mask)
73     to select the pool at each step. Defaults to _myopic_best_pool.
74
75 Returns (history, cleared_mask, utility).
76 """
77 n = len(p)
78 current_p = list(p)
79 cleared_mask = 0
80 history = ()
81
82 for _ in range(B):
83     if pool_selector is not None:
84         pool = pool_selector(current_p, u, G, n, cleared_mask)
85     else:
86         pool = _myopic_best_pool(current_p, u, G, n, cleared_mask)
87     if pool == 0:
88         break # no useful pool
89     r = test_result(pool, z_mask)
90     history = history + ((pool, r),)
91     if r == 0:
92         cleared_mask |= pool
93     current_p = bayesian_update_single_test(current_p, pool, r, n)
94
95 utility = sum(u[i] for i in indices_from_mask(cleared_mask, n))
96 return history, cleared_mask, utility
97
98
99 def greedy_myopic_expected_utility(p, u, B, G, pool_selector=None):
100     """Expected utility of the myopic greedy strategy.
101
102     At each step picks the myopic-best pool, then recurses over all
103     possible outcomes (r=0,...,|t|) weighted by their probabilities.
104
105     Parameters
106     -----
107     pool_selector : callable or None
108         If provided, called as pool_selector(p, u, G, n, cleared_mask)
109         to select the pool at each step. Defaults to _myopic_best_pool.
110
111     """
112     n = len(p)
113
114     def recurse(current_p, b, cleared_mask):
115         if b == 0:
116             return sum(u[i] for i in indices_from_mask(cleared_mask, n))
117
118         if pool_selector is not None:
119             pool = pool_selector(current_p, u, G, n, cleared_mask)
120         else:
121             pool = _myopic_best_pool(current_p, u, G, n, cleared_mask)
122         if pool == 0:
123             return sum(u[i] for i in indices_from_mask(cleared_mask, n))
124
125         pool_idx = indices_from_mask(pool, n)
126         pmf = _poisson_binomial_pmf([current_p[i] for i in pool_idx])
127
128         ev = 0.0
129         for r in range(len(pool_idx) + 1):
130             if pmf[r] < 1e-15:
131                 continue
132             new_p = bayesian_update_single_test(current_p, pool, r, n)
133             new_cleared = cleared_mask | pool if r == 0 else cleared_mask
134             ev += pmf[r] * recurse(new_p, b - 1, new_cleared)
135         return ev
136
137     return recurse(list(p), B, 0)
138
139 # -----

```

```

140 # Lookahead greedy: maximize total expected utility (not just immediate)
141 # -----
142
143 def _lookahead_best_pool(p, u, G, n, b, cleared_mask):
144     """Pick pool maximizing E[total utility over remaining b tests].
145
146     For each candidate pool, sums over all possible outcomes  $r=0,\dots,t$ ,
147     applies Bayesian update, and recursively evaluates future utility.
148     Returns (best_pool, expected_utility).
149     """
150     if b == 0:
151         return 0, sum(u[i] for i in indices_from_mask(cleared_mask, n))
152
153     pools = all_pools(n, G, include_empty=False)
154     best_pool, best_ev = 0, -1.0
155
156     for pool in pools:
157         pool_idx = indices_from_mask(pool, n)
158         pmf = _poisson_binomial_pmf([p[i] for i in pool_idx])
159
160         ev = 0.0
161         for r in range(len(pool_idx) + 1):
162             if pmf[r] < 1e-15:
163                 continue
164             new_p = bayesian_update_single_test(p, pool, r, n)
165             new_cleared = cleared_mask | pool if r == 0 else cleared_mask
166             # Recurse: for future steps, use MYOPIC selection (not full lookahead)
167             # Full lookahead at every level would be the optimal DP (too expensive).
168             _, future_val = _greedy_future(new_p, u, G, n, b - 1, new_cleared)
169             ev += pmf[r] * future_val
170
171             if ev > best_ev:
172                 best_ev = ev
173                 best_pool = pool
174
175     # Also consider doing nothing
176     nothing_val = sum(u[i] for i in indices_from_mask(cleared_mask, n))
177     if b > 0:
178         _, future_nothing = _greedy_future(p, u, G, n, b - 1, cleared_mask)
179         nothing_val = future_nothing
180     if nothing_val > best_ev:
181         best_ev = nothing_val
182         best_pool = 0
183
184     return best_pool, best_ev
185
186
187 def _greedy_future(p, u, G, n, b, cleared_mask):
188     """Evaluate expected utility of myopic greedy for remaining b tests."""
189     if b == 0:
190         val = sum(u[i] for i in indices_from_mask(cleared_mask, n))
191         return 0, val
192
193     pool = _myopic_best_pool(p, u, G, n, cleared_mask)
194     if pool == 0:
195         val = sum(u[i] for i in indices_from_mask(cleared_mask, n))
196         return 0, val
197
198     pool_idx = indices_from_mask(pool, n)
199     pmf = _poisson_binomial_pmf([p[i] for i in pool_idx])
200
201     ev = 0.0
202     for r in range(len(pool_idx) + 1):
203         if pmf[r] < 1e-15:
204             continue
205         new_p = bayesian_update_single_test(p, pool, r, n)
206         new_cleared = cleared_mask | pool if r == 0 else cleared_mask
207         _, future_val = _greedy_future(new_p, u, G, n, b - 1, new_cleared)
208         ev += pmf[r] * future_val
209
210     return pool, ev
211

```

```

212
213 def greedy_lookahead_simulate(p, u, B, G, z_mask):
214     """Simulate lookahead greedy on a fixed infection profile.
215
216     At step 1: uses full lookahead to pick the best pool.
217     At steps 2+: falls back to myopic (otherwise it's the full DP solver).
218
219     Returns (history, cleared_mask, utility).
220     """
221     n = len(p)
222     current_p = list(p)
223     cleared_mask = 0
224     history = ()
225
226     for step in range(B):
227         remaining = B - step
228         if step == 0:
229             pool, _ = _lookahead_best_pool(current_p, u, G, n, remaining, cleared_mask)
230         else:
231             pool = _myopic_best_pool(current_p, u, G, n, cleared_mask)
232
233         if pool == 0:
234             break
235         r = test_result(pool, z_mask)
236         history = history + ((pool, r),)
237         if r == 0:
238             cleared_mask |= pool
239         current_p = bayesian_update_single_test(current_p, pool, r, n)
240
241     utility = sum(u[i] for i in indices_from_mask(cleared_mask, n))
242     return history, cleared_mask, utility
243
244
245 # -----
246 # Counting-based greedy: full-history Bayesian update by counting
247 # -----
248
249 def greedy_myopic_counting_simulate(p, u, B, G, z_mask, pool_selector=None):
250     """Simulate myopic greedy with full-history Bayesian update by counting.
251
252     Like greedy_myopic_simulate but at each step computes posteriors by
253     enumerating all  $2^n$  infection profiles consistent with the FULL
254     history, rather than applying sequential single-test updates.
255
256     This uses bayesian_update_by_counting(p, history, n) which considers
257     all historical data  $h_k$  jointly.
258
259     Returns (history, cleared_mask, utility).
260     """
261     n = len(p)
262     cleared_mask = 0
263     history = ()
264
265     for _ in range(B):
266         # Compute posteriors from full history using counting
267         if history:
268             current_p = bayesian_update_by_counting(p, history, n)
269         else:
270             current_p = list(p)
271
272         if pool_selector is not None:
273             pool = pool_selector(current_p, u, G, n, cleared_mask)
274         else:
275             pool = _myopic_best_pool(current_p, u, G, n, cleared_mask)
276         if pool == 0:
277             break
278         r = test_result(pool, z_mask)
279         history = history + ((pool, r),)
280         if r == 0:
281             cleared_mask |= pool
282
283     utility = sum(u[i] for i in indices_from_mask(cleared_mask, n))

```

```

284     return history, cleared_mask, utility
285
286
287 def greedy_myopic_gibbs_simulate(p, u, B, G, z_mask,
288                                num_iterations=1000, burn_in=200, seed=None,
289                                pool_selector=None):
290     """Simulate myopic greedy with Gibbs sampling posterior updates.
291
292     Like greedy_myopic_counting_simulate but uses Gibbs sampling (MCMC)
293     to approximate posteriors instead of exact enumeration. Scales to
294     larger populations ( $n \sim 50+$ ) where counting ( $O(2^n)$ ) is infeasible.
295
296     Returns (history, cleared_mask, utility).
297     """
298     n = len(p)
299     cleared_mask = 0
300     history = ()
301
302     for _ in range(B):
303         # Compute posteriors from full history using Gibbs sampling
304         if history:
305             current_p = gibbs_update(p, history, n,
306                                     num_iterations=num_iterations,
307                                     burn_in=burn_in, seed=seed)
308         else:
309             current_p = list(p)
310
311         if pool_selector is not None:
312             pool = pool_selector(current_p, u, G, n, cleared_mask)
313         else:
314             pool = _myopic_best_pool(current_p, u, G, n, cleared_mask)
315         if pool == 0:
316             break
317         r = test_result(pool, z_mask)
318         history = history + ((pool, r),)
319         if r == 0:
320             cleared_mask |= pool
321
322     utility = sum(u[i] for i in indices_from_mask(cleared_mask, n))
323     return history, cleared_mask, utility
324
325
326 def greedy_myopic_gibbs_expected_utility(p, u, B, G,
327                                         num_iterations=1000, burn_in=200,
328                                         seed=42, pool_selector=None):
329     """Expected utility of myopic greedy with Gibbs sampling posteriors.
330
331     At each step picks the myopic-best pool using Gibbs-sampled posteriors,
332     then recurses over all possible outcomes weighted by their probabilities.
333     Uses a fixed seed for reproducibility within the recursive tree.
334     """
335     n = len(p)
336
337     def recurse(prior_p, history, b, cleared_mask):
338         if b == 0:
339             return sum(u[i] for i in indices_from_mask(cleared_mask, n))
340
341         # Compute posteriors from full history via Gibbs
342         if history:
343             current_p = gibbs_update(prior_p, history, n,
344                                     num_iterations=num_iterations,
345                                     burn_in=burn_in, seed=seed)
346         else:
347             current_p = list(prior_p)
348
349         if pool_selector is not None:
350             pool = pool_selector(current_p, u, G, n, cleared_mask)
351         else:
352             pool = _myopic_best_pool(current_p, u, G, n, cleared_mask)
353         if pool == 0:
354             return sum(u[i] for i in indices_from_mask(cleared_mask, n))
355

```

```

356     pool_idx = indices_from_mask(pool, n)
357     pmf = _poisson_binomial_pmf([current_p[i] for i in pool_idx])
358
359     ev = 0.0
360     for r in range(len(pool_idx) + 1):
361         if pmf[r] < 1e-15:
362             continue
363         new_cleared = cleared_mask | pool if r == 0 else cleared_mask
364         new_history = history + ((pool, r),)
365         ev += pmf[r] * recurse(prior_p, new_history, b - 1, new_cleared)
366     return ev
367
368     return recurse(list(p), (), B, 0)
369
370
371 def greedy_myopic_counting_expected_utility(p, u, B, G, pool_selector=None):
372     """Expected utility of myopic greedy with full-history Bayesian counting.
373
374     At each step picks the myopic-best pool using posteriors computed by
375     counting over all consistent worlds, then recurses over all possible
376     outcomes weighted by their probabilities.
377     """
378     n = len(p)
379
380     def recurse(prior_p, history, b, cleared_mask):
381         if b == 0:
382             return sum(u[i] for i in indices_from_mask(cleared_mask, n))
383
384         # Compute posteriors from full history
385         if history:
386             current_p = bayesian_update_by_counting(prior_p, history, n)
387         else:
388             current_p = list(prior_p)
389
390         if pool_selector is not None:
391             pool = pool_selector(current_p, u, G, n, cleared_mask)
392         else:
393             pool = _myopic_best_pool(current_p, u, G, n, cleared_mask)
394         if pool == 0:
395             return sum(u[i] for i in indices_from_mask(cleared_mask, n))
396
397         pool_idx = indices_from_mask(pool, n)
398         pmf = _poisson_binomial_pmf([current_p[i] for i in pool_idx])
399
400         ev = 0.0
401         for r in range(len(pool_idx) + 1):
402             if pmf[r] < 1e-15:
403                 continue
404             new_cleared = cleared_mask | pool if r == 0 else cleared_mask
405             new_history = history + ((pool, r),)
406             ev += pmf[r] * recurse(prior_p, new_history, b - 1, new_cleared)
407         return ev
408
409     return recurse(list(p), (), B, 0)

```

5 vw_demo.py

La “Variant of Greedy” (super-nodos de Francisco): muestra que la versión all-clear da lo mismo que el greedy miope, y el contraejemplo de que un escalar no ve la distribución del conteo.

```
1  """
2  VW super-node formulation – formulation check, counterexample, and
3  empirical comparison on n=6, G=3, B=2.
4
5  Setup. After k tests, the covered set S = u pools is built. V = N \ S.
6  The VW formulation augments the next-step problem with super-nodes
7  W = {w_T : T ⊆ S}, each carrying scalar (weight=|T|, prob, util), and
8  restricting the pool to use at most one w_T. We enumerate over W and
9  solve for V to pick the next pool.
10
11  Two natural readings of prob(w_T):
12  (A) all-clear:   prob_A(w_T) = ∏_{i∈T}(1 - p_i)
13  (B) any-positive: prob_B(w_T) = 1 - ∏_{i∈T}(1 - p_i)   (Codex assumption)
14
15  Empirical claims demonstrated here:
16  1. With prob_A and util_T = ∑ u_i, scalar VW EXACTLY reproduces the
17     standard myopic-greedy step value (utility-of-pool decomposes
18     additively, all-clear probability decomposes multiplicatively).
19  2. With prob_B, scalar VW solves a DIFFERENT objective and can pick
20     worse pools.
21  3. Both scalar variants are blind to the COUNT distribution of r > 0,
22     so neither closes the lookahead gap to the DP optimum. We exhibit
23     two T's with the same OR-prob but different count distributions –
24     any lookahead step will treat them differently.
25  """
26
27  import math
28  from itertools import combinations
29
30  from augmented.bayesian import (
31      _poisson_binomial_pmf,
32      bayesian_update_single_test,
33  )
34  from augmented.core import (
35      indices_from_mask,
36      mask_from_indices,
37      test_result,
38  )
39  from augmented.greedy import (
40      greedy_myopic_expected_utility,
41      greedy_myopic_simulate,
42  )
43  from augmented.solver import solve_optimal_dapts
44
45  def _pool_score_myopic(pool_mask, p, u):
46      idx = indices_from_mask(pool_mask)
47      if not idx:
48          return 0.0
49      util = sum(u[i] for i in idx)
50      prob_clear = 1.0
51      for i in idx:
52          prob_clear *= 1.0 - p[i]
53      return util * prob_clear
54
55  def _enumerate_full_pools(S_idx, V_idx, p, u, G):
56      universe = S_idx + V_idx
57      best_pool, best_val = 0, 0.0
58      for size in range(1, G + 1):
59          for combo in combinations(universe, size):
60              pool = mask_from_indices(combo)
61              val = _pool_score_myopic(pool, p, u)
62              if val > best_val:
63                  best_val, best_pool = val, pool
64      return best_pool, best_val
65
66
67
```

```

68
69 def _enumerate_vw(S_idx, V_idx, p, u, G, prob_model):
70     """Enumerate  $(T \subseteq S, U \subseteq V)$  with  $|T|+|U| \leq G$ ,  $|T \cap W| \leq 1$ .
71
72     Score = (util(U) + util(w_T)) · prob_clear(U) · prob(w_T)
73     where prob(w_T) is set per `prob_model` and util(w_T) =  $\sum_{i \in T} u_i$ .
74     """
75     best_pool, best_val = 0, 0.0
76     for t_size in range(0, min(len(S_idx), G) + 1):
77         for T in combinations(S_idx, t_size):
78             if T:
79                 util_T = sum(u[i] for i in T)
80                 pclear_T = 1.0
81                 for i in T:
82                     pclear_T *= 1.0 - p[i]
83                 if prob_model == "all_clear":
84                     prob_T = pclear_T
85                 elif prob_model == "or_event":
86                     prob_T = 1.0 - pclear_T
87                 else:
88                     raise ValueError(f"unknown prob_model {prob_model!r}")
89             else:
90                 util_T, prob_T = 0.0, 1.0
91
92             u_budget = G - t_size
93             for u_size in range(0, min(len(V_idx), u_budget) + 1):
94                 for U in combinations(V_idx, u_size):
95                     util_U = sum(u[i] for i in U)
96                     pclear_U = 1.0
97                     for i in U:
98                         pclear_U *= 1.0 - p[i]
99                     score = (util_U + util_T) * pclear_U * prob_T
100                     if score > best_val:
101                         best_val = score
102                         best_pool = mask_from_indices(list(U) + list(T))
103     return best_pool, best_val
104
105
106 def main():
107     n, G, B = 6, 3, 2
108     p_prior = [0.10, 0.15, 0.20, 0.08, 0.12, 0.25]
109     u = [4.0, 6.0, 3.0, 5.0, 7.0, 4.0]
110
111     print(f"Instance: n={n}, G={G}, B={B}")
112     print(f" prior p = {p_prior}")
113     print(f" util u = {u}")
114
115     # ----- Step 1 (fixed for reproducibility): test {0, 2, 4} -----
116     pool1 = mask_from_indices([0, 2, 4])
117     z_true = mask_from_indices([2])
118     r1 = test_result(pool1, z_true)
119     p_post = bayesian_update_single_test(p_prior, pool1, r1, n)
120     S_idx = indices_from_mask(pool1, n)
121     V_idx = [i for i in range(n) if i not in S_idx]
122
123     print(f"\nStep 1: pool={S_idx}, observed r={r1}")
124     print(f" posterior p = {[round(x, 4) for x in p_post]}")
125     print(f" S = {S_idx}, V = {V_idx}")
126
127     # ----- Step 2: three formulations of the myopic best-pool problem -----
128     pool_full, val_full = _enumerate_full_pools(S_idx, V_idx, p_post, u, G)
129     pool_A, val_A = _enumerate_vw(S_idx, V_idx, p_post, u, G, "all_clear")
130     pool_B, val_B = _enumerate_vw(S_idx, V_idx, p_post, u, G, "or_event")
131
132     print("\n--- Step-2 myopic decision: three formulations ---")
133     print(f" full pool enum. : pool={indices_from_mask(pool_full)} "
134           f"val={val_full:.6f}")
135     print(f" VW (all-clear) : pool={indices_from_mask(pool_A)} "
136           f"val={val_A:.6f}")
137     print(f" VW (or-event) : pool={indices_from_mask(pool_B)} "
138           f"val={val_B:.6f}")
139     print(f" val(VW-A) == val(full)? {abs(val_A - val_full) < 1e-9}")

```

```

140 print(f" val(VW-B) == val(full)? {abs(val_B - val_full) < 1e-9}")
141
142 # ----- DP optimum and true greedy value (end-to-end, B=2) -----
143 # The recursive greedy_myopic_expected_utility uses Poisson-Binomial PMF
144 # computed from MARGINAL posteriors, which is incorrect when Z|history is
145 # not independent. To get the truthful policy value we simulate greedy on
146 # every z-profile and weight by Pr(z) under the prior.
147 q = [1.0 - pi for pi in p_prior]
148 u_greedy_true = 0.0
149 for z in range(1 <= n):
150     w = 1.0
151     for i in range(n):
152         w *= p_prior[i] if (z >= i) & 1 else q[i]
153     _, _, util_z = greedy_myopic_simulate(p_prior, u, B, G, z)
154     u_greedy_true += w * util_z
155
156 u_greedy_recurse = greedy_myopic_expected_utility(p_prior, u, B, G)
157 u_opt, F_opt = solve_optimal_dapts(p_prior, u, B, G)
158
159 print("\n--- End-to-end expected utility (B=2) ---")
160 print(f" greedy (true, simulated) : {u_greedy_true:.6f}")
161 print(f" greedy (recursive estimate) : {u_greedy_recurse:.6f} "
162       "(inflated by indep. PMF approx)")
163 print(f" DP optimum : {u_opt:.6f}")
164 print(f" lookahead gap (DP - greedy) : "
165       f"{u_opt - u_greedy_true:.6f}")
166 print(" VW scalar formulation reproduces the myopic objective exactly")
167 print(" (when prob_A = all-clear), so it cannot shrink this gap on its own.")
168
169 # ----- Counterexample: OR-event compression loses count info -----
170 print("\n--- Counterexample: OR compression vs. count distribution ---")
171 p_a = 0.5
172 p_bc = 1 - 0.5 ** 0.5
173 or1 = 1 - (1 - p_a)
174 or2 = 1 - (1 - p_bc) ** 2
175 pmf1 = _poisson_binomial_pmf([p_a])
176 pmf2 = _poisson_binomial_pmf([p_bc, p_bc])
177 H1 = -sum(pi * math.log2(pi) for pi in pmf1 if pi > 0)
178 H2 = -sum(pi * math.log2(pi) for pi in pmf2 if pi > 0)
179 print(f" T1 = {{a}} p_a = {p_a:.4f}")
180 print(f" T2 = {{b, c}} p_b=p_c = {p_bc:.4f}")
181 print(f" OR-prob : T1 = {or1:.4f}, T2 = {or2:.4f} (EQUAL)")
182 print(f" all-clear prob: T1 = {1-or1:.4f}, T2 = {1-or2:.4f} (EQUAL)")
183 print(f" count PMF T1 : {[round(x, 4) for x in pmf1]}")
184 print(f" count PMF T2 : {[round(x, 4) for x in pmf2]}")
185 print(f" Shannon H(r) : T1 = {H1:.4f} bits, T2 = {H2:.4f} bits "
186       f"(diff = {H2-H1:.4f})")
187 print(" → T2 carries strictly more information about the joint Z than T1,")
188 print(" even though *both* scalar summaries (OR or all-clear) coincide.")
189 print(" Any scalar (weight, prob, util) encoding of w_T cannot")
190 print(" distinguish them; only the full count PMF can.")
191
192 # ----- Scaling: how does the lookahead gap grow with B? -----
193 print("\n--- Lookahead gap as a function of B (n=6, G=3) ---")
194 print(" B greedy(true) DP optimum gap")
195 for B_eval in (2, 3):
196     u_g = 0.0
197     q = [1.0 - pi for pi in p_prior]
198     for z in range(1 <= n):
199         w = 1.0
200         for i in range(n):
201             w *= p_prior[i] if (z >= i) & 1 else q[i]
202         _, _, util_z = greedy_myopic_simulate(p_prior, u, B_eval, G, z)
203         u_g += w * util_z
204     u_dp, _ = solve_optimal_dapts(p_prior, u, B_eval, G)
205     print(f" {B_eval} {u_g:11.6f} {u_dp:10.6f} {u_dp-u_g:+.6f}")
206 print(" VW scalar formulation is myopic by construction → it lives on")
207 print(" the 'greedy(true)' line. To close the gap, VW must carry the")
208 print(" full count PMF of w_T (i.e., a non-scalar super-node), at which")
209 print(" point it becomes equivalent to direct DP enumeration.")
210
211

```

```
212 if __name__ == "__main__":  
213     main()
```

6 vw_restrict.py

Cómo podar los muchos super-nodos posibles con un heurístico (partner) sin perder el bueno.

```
1 """
2 VW super-node restriction (Q3 from docs/notes/vw_submodularity.md).
3
4 Question. The VW framework enumerates  $W = \{w_T : T \subseteq S, T \neq \emptyset\}$ , which is
5  $2^{|S|-1}$  super-nodes. Can we restrict to a top-L subset using a cheap
6 ranking heuristic, without losing the optimal pool?
7
8 Approach. For each candidate T, the "real" value is
9  $val(T) = \max_{\{U \subseteq V, |U| \leq -G|T|\}} \sum_{u \in u_T} \prod_{i \in u_T} (1 - p_i)$ 
10 Computing val(T) is exactly the work we wanted to save. We measure
11  $L_{min}(rank) =$  the smallest L such that the top-L T's by 'rank' contain
12 an optimal one – i.e., a  $T^*$  with  $val(T^*) = \max_T val(T)$ .
13
14 Heuristics tested (each  $O(|T|)$ ):
15     self_score(T) =  $\sum (u_T) \cdot \prod (1 - p_T)$       myopic value of pool=T alone
16     prob(T)       =  $\prod (1 - p_T)$                     all-clear prob alone
17     util(T)       =  $\sum u_T$                           utility sum alone
18     random        =                                     sanity baseline
19
20 All heuristics keep the V-only pool ( $T = \emptyset$ ) implicitly available; we
21 only rank non-empty T's.
22
23 Empirical setup. Random p (low-prevalence) and u, simulate two
24 augmented tests to build S, then evaluate the restriction at step 3.
25 """
26
27 import math
28 import random
29 from itertools import combinations
30
31 from augmented.bayesian import (
32     _poisson_binomial_pmf,
33     bayesian_update_single_test,
34 )
35 from augmented.core import (
36     indices_from_mask,
37     mask_from_indices,
38 )
39
40 def _prod(xs):
41     out = 1.0
42     for x in xs:
43         out *= x
44     return out
45
46
47 def _pool_value(pool_idx, p, u):
48     if not pool_idx:
49         return 0.0
50     util = sum(u[i] for i in pool_idx)
51     pclear = _prod((1 - p[i]) for i in pool_idx)
52     return util * pclear
53
54
55 def _best_pool_with_T(T, V_idx, p, u, G):
56     """Best pool t = T ∪ U with U ⊆ V, |t| ≤ G."""
57     t_size = len(T)
58     if t_size > G:
59         return 0.0
60     u_budget = G - t_size
61     best_val = 0.0
62     u_min_size = 1 if not T else 0 # require non-empty pool overall
63     for u_size in range(u_min_size, min(len(V_idx), u_budget) + 1):
64         for U in combinations(V_idx, u_size):
65             val = _pool_value(list(T) + list(U), p, u)
66             if val > best_val:
67                 best_val = val
```

```

69     return best_val
70
71
72 def _entropy(pmf):
73     return -sum(pi * math.log2(pi) for pi in pmf if pi > 0)
74
75
76 def _best_v_partner_per_size(V_idx, p, u, G):
77     """Precompute, for each k = 0..G, the best V-pool of size exactly k by
78     its myopic value, returning (util_sum, prob_clear, val).
79
80     For k > |V|, returns the best of size |V|. The lookup gives a tight
81     surrogate for "what U could partner with a T of size -G|T|"."""
82     out = {0: (0.0, 1.0, 0.0)}
83     for k in range(1, G + 1):
84         best = (0.0, 0.0, 0.0)
85         if k <= len(V_idx):
86             for U in combinations(V_idx, k):
87                 util = sum(u[i] for i in U)
88                 pclear = _prod((1 - p[i]) for i in U)
89                 val = util * pclear
90                 if val > best[2]:
91                     best = (util, pclear, val)
92             else:
93                 best = out[len(V_idx)]
94         out[k] = best
95     return out
96
97
98 def restriction_experiment(p_post, S_idx, V_idx, u, G, rand_seed=0,
99                           lambdas=(1.0, 5.0, 25.0)):
100     """Compute L_min for each ranking heuristic on a fixed (p, S, V, u, G).
101
102     Heuristics evaluated:
103     self      :  $\sum(u_T) \cdot \prod(1-p_T)$           myopic value of pool=T alone
104     prob      :  $\prod(1-p_T)$                           all-clear prob
105     util      :  $\sum u_T$                                 utility sum
106     lent_     :  $\text{self} + \lambda \cdot H(r_T)$           entropy-augmented (one per  $\lambda$ )
107     partner   :  $\sum(u_T + u^*) \cdot \prod(1-p_T) \cdot p^*$  best V-partner of size -G|T|
108     rand      : uniform random                          sanity baseline
109
110     # All non-empty  $T \subseteq S$  with  $|T| \leq G$ 
111     all_T = []
112     for size in range(1, min(len(S_idx), G) + 1):
113         for combo in combinations(S_idx, size):
114             all_T.append(combo)
115
116     # Ground truth values
117     val_empty = _best_pool_with_T((), V_idx, p_post, u, G)
118     val_T = {T: _best_pool_with_T(T, V_idx, p_post, u, G) for T in all_T}
119     val_full = max([val_empty] + list(val_T.values()))
120
121     # Precompute best V partner per size (used by h_partner)
122     best_partner = _best_v_partner_per_size(V_idx, p_post, u, G)
123
124     # Per-T stats used by multiple heuristics
125     T_self = {}
126     T_entropy = {}
127     for T in all_T:
128         util_T = sum(u[i] for i in T)
129         prob_T = _prod((1 - p_post[i]) for i in T)
130         T_self[T] = util_T * prob_T
131         pmf = _poisson_binomial_pmf([p_post[i] for i in T])
132         T_entropy[T] = _entropy(pmf)
133
134     # Heuristics
135     def h_self(T):
136         return T_self[T]
137
138     def h_prob(T):
139         return _prod((1 - p_post[i]) for i in T)

```

```

141 def h_util(T):
142     return sum(u[i] for i in T)
143
144 def h_partner(T):
145     util_T = sum(u[i] for i in T)
146     prob_T = _prod((1 - p_post[i]) for i in T)
147     u_p, p_p, _ = best_partner[G - len(T)]
148     return (util_T + u_p) * prob_T * p_p
149
150 def make_h_entropy(lam):
151     def h(T):
152         return T_self[T] + lam * T_entropy[T]
153     return h
154
155 rng = random.Random(rand_seed)
156 rand_keys = {T: rng.random() for T in all_T}
157
158 def h_rand(T):
159     return rand_keys[T]
160
161 def L_min(rank_fn):
162     if val_empty >= val_full - 1e-12:
163         return 0
164     ranked = sorted(all_T, key=lambda T: -rank_fn(T))
165     best_so_far = val_empty
166     for L, T in enumerate(ranked, 1):
167         if val_T[T] > best_so_far:
168             best_so_far = val_T[T]
169         if best_so_far >= val_full - 1e-12:
170             return L
171     return len(ranked)
172
173 out = {
174     "W_size": len(all_T),
175     "val_full": val_full,
176     "val_empty": val_empty,
177     "L_min_self": L_min(h_self),
178     "L_min_prob": L_min(h_prob),
179     "L_min_util": L_min(h_util),
180     "L_min_partner": L_min(h_partner),
181     "L_min_rand": L_min(h_rand),
182 }
183 for lam in lambdas:
184     out[f"L_min_ent_{lam:g}"] = L_min(make_h_entropy(lam))
185 return out
186
187
188 def _sample_r(pool_idx, p, rng):
189     return sum(1 for i in pool_idx if rng.random() < p[i])
190
191
192 def adversarial_instance():
193     """Hand-crafted instance where self_score's L_min is large.
194
195     Mechanism: include in S a high-(util · prob_clear) BIG-T candidate
196     whose val(T) is mediocre because the prob-clear factor multiplies
197     multiplicatively with V's prob-clear, sinking the joint pool. The
198     truly optimal T is a *singleton* whose self_score is lower but whose
199     val(T) is higher because |T|=1 leaves more budget for V.
200
201     Returns (p_post, S_idx, V_idx, u, G).
202     """
203     n = 6
204     G = 4
205     # S = {0, 1, 2, 3}, V = {4, 5}
206     p_post = [0.01, 0.01, 0.60, 0.05, 0.05, 0.50]
207     u = [1.0, 1.0, 100.0, 50.0, 60.0, 80.0]
208     S_idx = [0, 1, 2, 3]
209     V_idx = [4, 5]
210     return p_post, S_idx, V_idx, u, G
211
212

```

```

213 def run_trial(p_prior, u, n, G, k_priors, trial_seed):
214     """Run k_priors random tests on (p_prior, u), then evaluate restriction.
215
216     Returns the experiment dict, or None if S ends up empty / |T| > G is
217     forced (degenerate).
218     """
219     rng = random.Random(trial_seed)
220     p_cur = list(p_prior)
221     S = 0
222     for _ in range(k_priors):
223         pool_idx = tuple(sorted(rng.sample(range(n), G)))
224         pool_mask = mask_from_indices(pool_idx)
225         r = _sample_r(pool_idx, p_cur, rng)
226         p_cur = bayesian_update_single_test(p_cur, pool_mask, r, n)
227         S |= pool_mask
228     S_idx = indices_from_mask(S)
229     V_idx = [i for i in range(n) if i not in S_idx]
230     if not S_idx:
231         return None
232     return restriction_experiment(p_cur, S_idx, V_idx, u, G,
233                                rand_seed=trial_seed)
234
235
236 def main():
237     n, G = 10, 4
238     K = 20
239
240     # Reproducible prior + utilities
241     rng_inst = random.Random(7)
242     p_prior = [rng_inst.uniform(0.05, 0.35) for _ in range(n)]
243     u = [rng_inst.uniform(1.0, 10.0) for _ in range(n)]
244
245     print(f"Setup: n={n}, G={G}, K={K} random 2-step histories")
246     print(f"  p_prior = {[round(x, 3) for x in p_prior]}")
247     print(f"  u       = {[round(x, 2) for x in u]}\n")
248
249     results = []
250     for trial in range(K):
251         rng = random.Random(1000 + trial)
252         # Two random pools of size G; observe r's via Bernoulli draws on prior
253         pool1_idx = tuple(sorted(rng.sample(range(n), G)))
254         pool1 = mask_from_indices(pool1_idx)
255         r1 = _sample_r(pool1_idx, p_prior, rng)
256         p1 = bayesian_update_single_test(p_prior, pool1, r1, n)
257
258         pool2_idx = tuple(sorted(rng.sample(range(n), G)))
259         pool2 = mask_from_indices(pool2_idx)
260         r2 = _sample_r(pool2_idx, p1, rng)
261         p2 = bayesian_update_single_test(p1, pool2, r2, n)
262
263         S = pool1 | pool2
264         S_idx = indices_from_mask(S)
265         V_idx = [i for i in range(n) if i not in S_idx]
266         if not S_idx:
267             continue
268
269         res = restriction_experiment(p2, S_idx, V_idx, u, G, rand_seed=trial)
270         results.append(res)
271
272     keys = sorted({k for k in results[0] if k.startswith("L_min_")})
273     K_eff = len(results)
274     means = {k: sum(r[k] for r in results) / K_eff for k in keys}
275     maxes = {k: max(r[k] for r in results) for k in keys}
276     mean_W = sum(r["W_size"] for r in results) / K_eff
277     print(f"Summary over {K_eff} trials:   mean |W| = {mean_W:.1f}")
278     print(f" heuristic           mean L_min   max L_min   "
279           f"mean / |W|")
280     for k in keys:
281         label = k.replace("L_min_", "")
282         ratio = means[k] / mean_W if mean_W > 0 else 0.0
283         print(f" {label:<20} {means[k]:>10.2f} {maxes[k]:>9} "
284               f"{ratio:>10.3f}")

```

```

285
286 # Hand-crafted adversarial
287 print("\n--- Adversarial instance (self_score worst-case demo) ---")
288 p_adv, S_adv, V_adv, u_adv, G_adv = adversarial_instance()
289 print(f" S = {S_adv}, V = {V_adv}, G = {G_adv}")
290 print(f" p = {p_adv}")
291 print(f" u = {u_adv}")
292 res_adv = restriction_experiment(p_adv, S_adv, V_adv, u_adv, G_adv,
293                                rand_seed=0)
294 keys_adv = sorted([k for k in res_adv if k.startswith("L_min_")])
295 print(f" |W| = {res_adv['W_size']}    val_full = {res_adv['val_full']:.4f}"
296       f"    val_empty = {res_adv['val_empty']:.4f}")
297 print(f" heuristic          L_min    L_min / |W|")
298 for k in keys_adv:
299     label = k.replace("L_min_", "")
300     ratio = res_adv[k] / res_adv['W_size'] if res_adv['W_size'] else 0.0
301     print(f" {label:<20}    {res_adv[k]:>5}    {ratio:>10.3f}")
302
303
304 if __name__ == "__main__":
305     main()

```

7 rl_examples.py

Primer puente a RL: value iteration (coincide con el DP) y Q-learning tabular (se acerca al óptimo en casos chicos).

```
1 """
2 Augmented DAPTS as a finite-horizon Markov Decision Process (MDP),
3 with two pedagogical "first RL examples":
4
5 1. value_iteration(p, u, B, G):
6     Explicit Bellman value iteration (backward sweep) over enumerated
7     information states. Mathematically identical to the existing DP
8     solver in augmented/solver.py, but written in MDP vocabulary:
9     states, actions, transitions, rewards, Bellman equation. Intended
10    as the first bridge from "dynamic programming over trees" to
11    "value iteration on an MDP".
12
13 2. tabular_q_learning(p, u, B, G, ...):
14     Classic Q-learning with  $\epsilon$ -greedy exploration. Samples episodes
15     (one ground-truth infection profile per episode), interacts with
16     the DAPTS environment, and updates Q(state, action) via TD(0).
17     Converges to the Q* produced by value_iteration.
18
19 MDP formulation
20 -----
21 State s = (k, remaining, cleared_mask)
22 * k:          number of tests already used (0..B)
23 * remaining:   frozenset of infection profiles z consistent with
24               the observed history
25 * cleared_mask: bitmask of individuals proven healthy so far
26 Action a = pool mask (subset of [n] of size <= G; a = 0 means "wait")
27 Transition: choosing pool a when state is (k, remaining, cleared)
28 * Observe r = |a ∩ Z|, with Z ~ prior restricted to `remaining`.
29 * Next remaining = { z ∈ remaining : test_result(a, z) = r }.
30 * Next cleared = cleared ∪ a if r = 0 else cleared.
31 Reward: 0 at all non-terminal steps; at step B, reward = sum_{i in cleared} u_i.
32 Objective: maximize expected terminal reward.
33
34 Under this formulation, V*(initial_state) equals the optimal DAPTS
35 expected utility, which matches solve_optimal_dapts exactly.
36 """
37
38 import random
39
40 from augmented.core import all_pools, test_result
41
42 # -----
43 # MDP helpers
44 # -----
45
46 def _prior_weights(p, n):
47     """Return w[z] = Pr(Z = z) under the independent prior for all z."""
48     q = [1.0 - pi for pi in p]
49     num_profiles = 1 << n
50     w = [0.0] * num_profiles
51     for z in range(num_profiles):
52         wz = 1.0
53         for i in range(n):
54             wz *= p[i] if (z >> i & 1) else q[i]
55         w[z] = wz
56     return w
57
58
59 def _cleared_utility(cleared_mask, u, n):
60     """Total utility of individuals marked cleared."""
61     return sum(u[i] for i in range(n) if (cleared_mask >> i & 1))
62
63
64 def _transition(remaining, cleared, pool, w):
65     """Partition a transition by observed outcome r.
66
67     Returns a list of (r, mass_r, new_remaining, new_cleared) entries
```

```

69 with mass_r = sum of prior weights of profiles in `remaining` that
70 would produce outcome r when pool is tested.
71 """
72 buckets = {}
73 for z in remaining:
74     r = test_result(pool, z)
75     buckets.setdefault(r, []).append(z)
76
77 out = []
78 for r, zs in buckets.items():
79     mass_r = sum(w[z] for z in zs)
80     new_cleared = cleared | pool if r == 0 else cleared
81     out.append((r, mass_r, frozenset(zs), new_cleared))
82 return out
83
84
85 # -----
86 # Example 1: Value iteration on the information-state MDP
87 # -----
88
89 def value_iteration(p, u, B, G):
90     """Bellman value iteration (= backward induction) over enumerated
91     information states.
92
93     Because the horizon is finite and every state belongs to exactly
94     one step k, the MDP admits a single backward sweep, so "value
95     iteration" here is really finite-horizon backward induction.
96
97     Returns
98     -----
99     V : dict
100         (k, remaining, cleared) -> V*(state), the optimal value.
101     Q : dict
102         (k, remaining, cleared, pool) -> Q*(state, action).
103     pi : dict
104         (k, remaining, cleared) -> argmax pool.
105
106     Notes
107     -----
108     Values are CONDITIONAL: V(s) is the expected future utility given
109     the agent is at s. In particular V*(initial_state) is the optimal
110     DAPTS value, matching solve_optimal_dapts to machine precision.
111     """
112     n = len(p)
113     w = _prior_weights(p, n)
114     pools = all_pools(n, G, include_empty=True)
115     all_z = frozenset(range(1 << n))
116
117     # ----- Forward reach: enumerate states reachable at each step -----
118     states_at = [set() for _ in range(B + 1)]
119     states_at[0].add((all_z, 0))
120     for k in range(B):
121         for remaining, cleared in states_at[k]:
122             for a in pools:
123                 for _r, _m, new_remaining, new_cleared in \
124                     _transition(remaining, cleared, a, w):
125                     states_at[k + 1].add((new_remaining, new_cleared))
126
127     # ----- Backward Bellman sweep -----
128     V, Q, pi = {}, {}, {}
129
130     # Terminal layer: V = utility of individuals cleared so far
131     for remaining, cleared in states_at[B]:
132         V[(B, remaining, cleared)] = _cleared_utility(cleared, u, n)
133
134     # Bellman backup for k = B-1, B-2, ..., 0
135     for k in range(B - 1, -1, -1):
136         for remaining, cleared in states_at[k]:
137             total_mass = sum(w[z] for z in remaining)
138             best_val = -float('inf')
139             best_a = 0

```

```

141     for a in pools:
142         ev = 0.0
143         for _r, mass_r, new_rem, new_cl in \
144             _transition(remaining, cleared, a, w):
145             prob_r = mass_r / total_mass if total_mass > 0 else 0.0
146             ev += prob_r * V[(k + 1, new_rem, new_cl)]
147
148         Q[(k, remaining, cleared, a)] = ev
149         if ev > best_val:
150             best_val = ev
151             best_a = a
152
153         V[(k, remaining, cleared)] = best_val
154         pi[(k, remaining, cleared)] = best_a
155
156     return V, Q, pi
157
158
159 def value_iteration_optimal_value(p, u, B, G):
160     """Convenience wrapper: V*(initial) as a single float."""
161     V, _, _ = value_iteration(p, u, B, G)
162     n = len(p)
163     initial = (0, frozenset(range(1 << n)), 0)
164     return V[initial]
165
166
167 # -----
168 # Example 2: Tabular Q-learning (model-free) on the same MDP
169 # -----
170
171 def _sample_profile(p, rng):
172     z = 0
173     for i, pi in enumerate(p):
174         if rng.random() < pi:
175             z |= 1 << i
176     return z
177
178
179 def tabular_q_learning(p, u, B, G, num_episodes=5000, alpha='auto',
180                       epsilon=0.3, seed=0):
181     """Tabular Q-learning with  $\epsilon$ -greedy exploration.
182
183     No knowledge of the transition/reward model is assumed: the agent
184     only interacts with the DAPTS environment episode by episode. After
185     enough episodes,  $Q(\text{state}, \text{action})$  approaches the  $Q^*$  returned by
186     value_iteration.
187
188     Parameters
189     -----
190     num_episodes : int
191         Number of training episodes (each samples one Z and plays B tests).
192     alpha : float or 'auto'
193         Step size. If 'auto', uses the Robbins-Monro schedule
194          $\alpha = 1 / (1 + N(s,a))$ , where  $N(s,a)$  is the visit count INCLUDING
195         the current visit (incremented before the step-size is read).
196         This schedule guarantees convergence of tabular Q-learning.
197         Fixed  $\alpha$  (e.g. 0.1) runs the usual "slow average" variant but
198         does not converge to  $Q^*$  exactly.
199     epsilon : float
200         Exploration probability for  $\epsilon$ -greedy action selection.
201     seed : int
202         RNG seed.
203
204     Returns
205     -----
206     Q : dict
207         (state, action) -> estimated Q value. Only (state, action) pairs
208         actually visited during training are present.
209     """
210     n = len(p)
211     rng = random.Random(seed)
212     pools = all_pools(n, G, include_empty=True)

```

```

213 all_z = frozenset(range(1 << n))
214
215 Q = {}
216 N = {} # visit counts, used only when alpha == 'auto'
217
218 def q_get(s, a):
219     return Q.get((s, a), 0.0)
220
221 def greedy_action(s):
222     best_a = pools[0]
223     best_v = q_get(s, best_a)
224     for a in pools[1:]:
225         v = q_get(s, a)
226         if v > best_v:
227             best_v = v
228             best_a = a
229     return best_a
230
231 def step_size(s, a):
232     if alpha == 'auto':
233         N[(s, a)] = N.get((s, a), 0) + 1
234         return 1.0 / (1.0 + N[(s, a)])
235     return alpha
236
237 for _ in range(num_episodes):
238     z_true = _sample_profile(p, rng)
239     remaining = all_z
240     cleared = 0
241
242     for k in range(B):
243         s = (k, remaining, cleared)
244
245         # ε-greedy
246         if rng.random() < epsilon:
247             a = rng.choice(pools)
248         else:
249             a = greedy_action(s)
250
251         # Environment step (the true z is fixed for this episode)
252         r = test_result(a, z_true)
253         new_remaining = frozenset(z for z in remaining
254                                   if test_result(a, z) == r)
255         new_cleared = cleared | a if r == 0 else cleared
256         s_next = (k + 1, new_remaining, new_cleared)
257
258         # TD target: at terminal, use the utility of cleared_mask;
259         # otherwise, bootstrap with max_a' Q(s', a').
260         if k == B - 1:
261             target = _cleared_utility(new_cleared, u, n)
262         else:
263             target = max(q_get(s_next, aa) for aa in pools)
264
265         # Q-learning update
266         a_step = step_size(s, a)
267         Q[(s, a)] = q_get(s, a) + a_step * (target - q_get(s, a))
268
269         remaining, cleared = new_remaining, new_cleared
270
271     return Q
272
273
274 def q_learning_policy_value(p, u, B, G, Q):
275     """Evaluate the greedy policy extracted from Q by full enumeration.
276
277     Runs the Bellman equation once using the policy a(s) = argmax_a Q(s,a)
278     (with deterministic tie-breaking). Returns the expected utility at
279     the initial state – comparable to value_iteration_optimal_value.
280     """
281     n = len(p)
282     w = _prior_weights(p, n)
283     pools = all_pools(n, G, include_empty=True)
284     all_z = frozenset(range(1 << n))

```

```

285
286 def policy_at(s):
287     best_a, best_v = pools[0], Q.get((s, pools[0]), 0.0)
288     for a in pools[1:]:
289         v = Q.get((s, a), 0.0)
290         if v > best_v:
291             best_v, best_a = v, a
292     return best_a
293
294 memo = {}
295
296 def value(k, remaining, cleared):
297     key = (k, remaining, cleared)
298     if key in memo:
299         return memo[key]
300
301     if k == B:
302         memo[key] = _cleared_utility(cleared, u, n)
303         return memo[key]
304
305     a = policy_at(key)
306     total_mass = sum(w[z] for z in remaining)
307     ev = 0.0
308     for _r, mass_r, new_rem, new_cl in \
309         _transition(remaining, cleared, a, w):
310         prob_r = mass_r / total_mass if total_mass > 0 else 0.0
311         ev += prob_r * value(k + 1, new_rem, new_cl)
312
313     memo[key] = ev
314     return ev
315
316 return value(0, all_z, 0)

```

8 rl_env.py

Dos entornos Gym para RL: uno exacto (para validar contra el DP, n chico) y uno con cubetas (para escalar a N grande).

```
1  """
2  Gymnasium environments for augmented DAPTS, ready for reinforcement learning.
3
4  Both environments are powered entirely by the ``augmented`` package (Bayesian
5  updates and myopic-greedy pool selection) -- there is NO MOSEK / conic-solver
6  dependency, unlike the original ``classical/rl_training`` code.
7
8  DaptsExactEnv    Exact belief-state MDP for SMALL n. The observation is the
9                  full posterior over the 2**n infection profiles, so a trained
10                 policy can in principle match the DP optimum
11                 (augmented.solver.solve_optimal_dapts). Use it to VALIDATE
12                 that RL recovers the optimum.
13
14  DaptsBucketEnv   Bucketed environment for LARGE N (beyond the n<=14 wall of
15                 exact DP). It is a port of
16                 ``classical/rl_training/PP0_bucket_gymnasium_B*.py``: the
17                 observation is a fixed-size histogram of agents by
18                 (health bucket, utility bucket). The RL agent chooses the
19                 FIRST pool; the remaining B-1 tests are played by augmented
20                 myopic greedy.
21
22  Reproducibility
23  -----
24  Every stochastic choice goes through the env's gymnasium ``np_random``, seeded
25  via ``reset(seed=...)``. Instance samplers receive that same Generator, so a
26  fixed seed fully determines an episode (instance, true profile, shuffles).
27  """
28
29  import numpy as np
30  import gymnasium as gym
31  from gymnasium import spaces
32
33  from augmented.core import all_pools, indices_from_mask, mask_from_indices, test_result
34  from augmented.bayesian import bayesian_update_single_test
35  from augmented.greedy import _myopic_best_pool, greedy_myopic_simulate
36
37
38  # -----
39  # Helpers
40  # -----
41
42  def prior_profile_weights(p):
43      """Return w[z] = Pr(Z = z) under the independent prior, for all 2**n z."""
44      n = len(p)
45      w = np.ones(1 << n, dtype=np.float64)
46      for z in range(1 << n):
47          prob = 1.0
48          for i in range(n):
49              prob *= p[i] if (z >> i) & 1 else (1.0 - p[i])
50          w[z] = prob
51      return w
52
53
54  def sample_profile(p, rng):
55      """Sample a true infection profile z (bitmask) from independent prior p."""
56      z = 0
57      for i in range(len(p)):
58          if rng.random() < p[i]:
59              z |= 1 << i
60      return z
61
62
63  # -----
64  # Exact belief-state environment (validation against DP)
65  # -----
66
67  class DaptsExactEnv(gym.Env):
```

```

68 """Exact belief-MDP for augmented DAPTS on small n.
69
70 State (observation): the full posterior over the 2*n profiles, plus a
71 cleared-individuals indicator and the fraction of budget remaining. This
72 is the *sufficient statistic* of the problem, so an optimal policy on this
73 env attains the DP optimum.
74
75 Parameters
76 -----
77 instance_sampler : callable
78     ``instance_sampler(rng) -> (p, u)`` where p, u are length-n sequences.
79     For a fixed instance, return the same (p, u) ignoring rng.
80 B, G, n : int
81     Budget (tests), pool size, population size.
82 max_n : int
83     Guard rail; the observation has 2*n entries.
84 """
85
86 metadata = {"render_modes": []}
87
88 def __init__(self, instance_sampler, B, G, n, max_n=10):
89     super().__init__()
90     if n > max_n:
91         raise ValueError(
92             f"DaptsExactEnv is for small n (got n={n} > max_n={max_n}); "
93             "use DaptsBucketEnv to scale.")
94     self.instance_sampler = instance_sampler
95     self.B, self.G, self.n = B, G, n
96     self.num_profiles = 1 << n
97     self.pools = all_pools(n, G, include_empty=False)
98
99     self.action_space = spaces.Discrete(len(self.pools))
100     obs_dim = self.num_profiles + n + 1
101     self.observation_space = spaces.Box(
102         low=0.0, high=1.0, shape=(obs_dim,), dtype=np.float32)
103
104     self.p = self.u = None
105     self._belief = None
106     self._cleared = 0
107     self._k = 0
108     self._z = 0
109
110 def reset(self, *, seed=None, options=None):
111     super().reset(seed=seed)
112     p, u = self.instance_sampler(self.np_random)
113     self.p, self.u = list(p), list(u)
114     if len(self.p) != self.n:
115         raise ValueError(
116             f"instance_sampler produced n={len(self.p)}, env expects {self.n}")
117
118     self._belief = prior_profile_weights(self.p)
119     # The true profile may be forced (used by the exact evaluator).
120     if options is not None and "force_z" in options:
121         self._z = int(options["force_z"])
122     else:
123         self._z = int(self.np_random.choice(self.num_profiles,
124                                             p=self._belief))
125
126     self._cleared = 0
127     self._k = 0
128     return self._obs(), {}
129
130 def _obs(self):
131     cleared = np.array([(self._cleared >> i) & 1 for i in range(self.n)],
132                        dtype=np.float32)
133     remaining = np.array([1.0 - self._k / self.B], dtype=np.float32)
134     return np.concatenate(
135         [self._belief.astype(np.float32), cleared, remaining])
136
137 def step(self, action):
138     pool = self.pools[int(action)]
139     r = test_result(pool, self._z)

```

```

140     # Posterior update: keep only profiles consistent with observed r.
141     keep = np.fromiter(
142         (1.0 if test_result(pool, z) == r else 0.0
143          for z in range(self.num_profiles)),
144         dtype=np.float64, count=self.num_profiles)
145     self._belief = self._belief * keep
146     total = self._belief.sum()
147     if total > 0:
148         self._belief /= total
149
150     if r == 0:
151         self._cleared |= pool
152     self._k += 1
153
154     done = self._k >= self.B
155     reward = 0.0
156     if done:
157         reward = float(sum(self.u[i] for i in range(self.n)
158                           if (self._cleared >> i) & 1))
159     return self._obs(), reward, done, False, {}
160
161
162 # -----
163 # Bucketed environment (scaling beyond the exact-DP wall)
164 # -----
165
166 class DaptsBucketEnv(gym.Env):
167     """Bucketed environment for large populations.
168
169     Port of ``classical/rl_training/PP0_bucket_gymnasium_B*.py`` with the
170     conic/MOSEK machinery replaced by augmented myopic greedy:
171
172     * Observation: histogram of the N agents over
173       (health bucket x utility bucket), plus the running utility sum and
174       health product of the agents picked so far. Fixed size -> scales in N.
175     * Action: pick an agent from a (health, utility) category, or STOP.
176       The RL agent assembles the FIRST pool (up to G agents).
177     * Reward: a true profile z is sampled; test 1 is the RL pool; tests
178       2..B are played by augmented myopic greedy. Reward = utility of all
179       individuals proven healthy.
180
181     Parameters
182     -----
183     instance_sampler : callable
184         ``instance_sampler(rng) -> (p, u)`` with length-N sequences;
185         p[i] = infection probability, u[i] = utility.
186     B, G, N : int
187         Budget, pool size, population size.
188     health_bins, utility_bins : int
189         Discretisation of the (health, utility) observation.
190     """
191
192     metadata = {"render_modes": []}
193
194     def __init__(self, instance_sampler, B, G, N,
195                 health_bins=4, utility_bins=3):
196         super().__init__()
197         self.instance_sampler = instance_sampler
198         self.B, self.G, self.N = B, G, N
199         self.health_bins = health_bins
200         self.utility_bins = utility_bins
201         self.num_categories = health_bins * utility_bins
202
203         self.observation_space = spaces.Box(
204             low=0.0, high=float(N),
205             shape=(self.num_categories + 2,), dtype=np.float32)
206         self.action_space = spaces.Discrete(self.num_categories + 1)
207
208         self.health_bin_edges = np.linspace(0.0, 1.0, health_bins + 1)
209         self.utility_bin_edges = np.array([0, 2, 3])
210
211         self.p = self.u = None

```

```

212     self.category_agents = {}
213     self.category_counts = {}
214     self.selected = []
215     self.attempts = 0
216     self.last_z = 0          # true profile of the most recent episode
217
218     def _category(self, health, utility):
219         ub = int(np.digitize(utility, self.utility_bin_edges) - 1)
220         ub = min(max(ub, 0), self.utility_bins - 1)
221         hb = int(np.digitize(health, self.health_bin_edges) - 1)
222         hb = min(max(hb, 0), self.health_bins - 1)
223         return ub * self.health_bins + hb
224
225     def reset(self, *, seed=None, options=None):
226         super().reset(seed=seed)
227         p, u = self.instance_sampler(self.np_random)
228         self.p = np.asarray(p, dtype=np.float64)
229         self.u = np.asarray(u, dtype=np.float64)
230
231         self.category_agents = {c: [] for c in range(self.num_categories)}
232         health = 1.0 - self.p
233         for i in range(self.N):
234             cat = self._category(health[i], self.u[i])
235             self.category_agents[cat].append(i)
236         for c in self.category_agents:
237             self.np_random.shuffle(self.category_agents[c])
238         self.category_counts = {c: len(self.category_agents[c])
239                                for c in range(self.num_categories)}
240         self.selected = []
241         self.attempts = 0
242         return self._obs(), {}
243
244     def _obs(self):
245         counts = np.array([self.category_counts[c]
246                           for c in range(self.num_categories)],
247                           dtype=np.float32)
248         usum = float(sum(self.u[i] for i in self.selected))
249         hprod = (float(np.prod([1.0 - self.p[i] for i in self.selected])))
250                 if self.selected else 1.0)
251         return np.concatenate([counts, [usum, hprod]]).astype(np.float32)
252
253     def step(self, action):
254         action = int(action)
255         done = False
256         if action == self.num_categories:          # STOP
257             done = True
258         elif self.category_counts.get(action, 0) > 0:
259             agent = self.category_agents[action].pop()
260             self.category_counts[action] -= 1
261             self.selected.append(agent)
262
263         self.attempts += 1
264         if self.attempts >= self.G:
265             done = True
266
267         reward = 0.0
268         if done:
269             reward = self._rollout_reward()
270         return self._obs(), reward, done, False, {}
271
272     def _rollout_reward(self):
273         """Sample z, play test 1 = RL pool, tests 2..B = augmented greedy."""
274         z = sample_profile(self.p, self.np_random)
275         self.last_z = z
276         rl_pool = mask_from_indices(self.selected) if self.selected else 0
277
278         calls = {"k": 0}
279
280         def selector(p, u, G, n, cleared_mask):
281             first = calls["k"] == 0
282             calls["k"] += 1
283             if first and rl_pool != 0:

```

```
284         return rl_pool
285     return _myopic_best_pool(p, u, G, n, cleared_mask)
286
287     _, _, util = greedy_myopic_simulate(
288         list(self.p), list(self.u), self.B, self.G, z,
289         pool_selector=selector)
290     return float(util)
```

9 rl_train.py

Entrena PPO de forma reproducible (con semilla), carga instancias (aleatorias o de CSV), y compara PPO contra el óptimo DP.

```
1  """
2  Reproducible PPO training and evaluation for the augmented DAPTS RL
3  environments (augmented/rl_env.py).
4
5  This is the augmented-package counterpart of
6  ``classical/rl_training/PP0_bucket_gymnasium_B*.py``: same PPO / Gymnasium
7  recipe, but powered by augmented Bayesian updates and myopic greedy (no MOSEK),
8  fully seeded, and with an exact-vs-DP validation path.
9
10 Instance sources
11 -----
12 random : (p, u) drawn from a fixed-seed RNG -> 100% reproducible.
13 csv     : parsed from a classical data CSV (classical/data/data_N*_*.csv);
14           agent tuples are (id, utility, health), so p = 1 - health.
15
16 Usage
17 -----
18 # Validate: does PPO match the DP optimum on a small instance?
19 python -m augmented.rl_train exact --source random --n 6 --B 2 --G 3 \
20         --timesteps 60000 --seed 0
21 python -m augmented.rl_train exact --source csv \
22         --csv classical/data/data_N3_d2_B2_G3.csv --timesteps 60000
23
24 # Scale: train the bucketed env beyond the exact-DP wall and beat greedy.
25 python -m augmented.rl_train bucket --source random --N 50 --B 2 --G 3 \
26         --timesteps 200000 --seed 0
27
28 Every run prints the seed and the instances used so results can be reproduced.
29 """
30
31 import argparse
32 import ast
33 import csv
34 import os
35 import re
36
37 import numpy as np
38
39 from augmented.core import mask_from_indices
40 from augmented.greedy import _myopic_best_pool, greedy_myopic_simulate
41 from augmented.solver import solve_optimal_dapts
42 from augmented.rl_env import DaptsBucketEnv, DaptsExactEnv, prior_profile_weights
43
44 DEFAULT_MODEL_DIR = os.path.join(os.path.dirname(__file__), "rl_models")
45
46 # -----
47 # Instance sources
48 # -----
49
50 def random_instance_sampler(n, util_choices=(1.0, 2.0, 3.0),
51                             p_low=0.0, p_high=1.0):
52     """A sampler that draws a fresh random (p, u) on each call.
53
54     Reproducible: the env feeds in its seeded gymnasium Generator.
55     """
56     util_choices = np.asarray(util_choices, dtype=float)
57
58     def sampler(rng):
59         p = rng.uniform(p_low, p_high, size=n)
60         u = rng.choice(util_choices, size=n)
61         return p.tolist(), u.tolist()
62
63     return sampler
64
65 def fixed_instance_sampler(p, u):
```

```

68 """A sampler that always returns the same (p, u) -- ignores the RNG."""
69 p, u = list(p), list(u)
70
71 def sampler(rng):
72     return list(p), list(u)
73
74 return sampler
75
76
77 def load_csv_instances(path):
78     """Parse a classical data CSV.
79
80     Returns (instances, N, B, G) where instances is a list of (p, u) with
81     p = 1 - health and u = utility. N, B, G come from the file name.
82     """
83     base = os.path.basename(path)
84     m = re.search(r"N(\d+)_d\d+_B(\d+)_G(\d+)", base)
85     if not m:
86         raise ValueError(f"cannot parse N, B, G from filename: {base}")
87     N, B, G = (int(x) for x in m.groups())
88
89     instances = []
90     with open(path, newline="") as fh:
91         for row in csv.DictReader(fh):
92             agents = sorted(ast.literal_eval(row["agents"]), key=lambda a: a[0])
93             p = [1.0 - float(a[2]) for a in agents]
94             u = [float(a[1]) for a in agents]
95             instances.append((p, u))
96     return instances, N, B, G
97
98
99 def csv_instance_sampler(instances):
100     """A sampler that picks one (p, u) uniformly from a list of instances."""
101     def sampler(rng):
102         p, u = instances[int(rng.integers(len(instances)))]
103         return list(p), list(u)
104
105     return sampler
106
107
108 # -----
109 # Shared rollout (matches DaptsBucketEnv._rollout_reward exactly)
110 # -----
111
112 def rollout_utility(p, u, B, G, z, first_pool=None):
113     """Cleared-utility of: test 1 = first_pool (or greedy), tests 2..B = greedy.
114
115     With first_pool=None this is pure augmented myopic greedy -- the baseline.
116     """
117     if not first_pool:
118         _, _, util = greedy_myopic_simulate(p, u, B, G, z)
119         return float(util)
120
121     calls = {"k": 0}
122
123     def selector(pp, uu, GG, nn, cleared):
124         first = calls["k"] == 0
125         calls["k"] += 1
126         if first:
127             return first_pool
128         return _myopic_best_pool(pp, uu, GG, nn, cleared)
129
130     _, _, util = greedy_myopic_simulate(p, u, B, G, z, pool_selector=selector)
131     return float(util)
132
133
134 # -----
135 # Training
136 # -----
137
138 def train(env, timesteps, seed, model_path=None, verbose=0):
139     """Train PPO on a Gymnasium env. Seeded for reproducibility."""

```

```

140 from stable_baselines3 import PPO
141
142 model = PPO("MlpPolicy", env, seed=seed, verbose=verbose)
143 model.learn(total_timesteps=timesteps)
144 if model_path:
145     os.makedirs(os.path.dirname(model_path) or ".", exist_ok=True)
146     model.save(model_path)
147     print(f" model saved to {model_path}.zip")
148 return model
149
150
151 # -----
152 # Evaluation
153 # -----
154
155 def evaluate_exact_vs_dp(model, p, u, B, G):
156     """Exact expected utility of the PPO policy vs the DP optimum.
157
158     The PPO policy value is computed exactly: enumerate every infection
159     profile z, play the policy deterministically, weight by Pr(z).
160     """
161     n = len(p)
162     env = DaptsExactEnv(fixed_instance_sampler(p, u), B, G, n)
163     weights = prior_profile_weights(p)
164
165     policy_value = 0.0
166     for z in range(1 << n):
167         if weights[z] == 0.0:
168             continue
169         obs, _ = env.reset(seed=12345, options={"force_z": z})
170         done = False
171         episode = 0.0
172         while not done:
173             action, _ = model.predict(obs, deterministic=True)
174             obs, reward, done, _, _ = env.step(action)
175             episode += reward
176             policy_value += weights[z] * episode
177
178     dp_value, _ = solve_optimal_dapts(p, u, B, G)
179     return policy_value, dp_value
180
181
182 def evaluate_bucket(model, instance_sampler, B, G, N, n_episodes, seed):
183     """Mean PPO reward vs mean greedy reward over matched episodes.
184
185     Each episode uses the same instance and same true profile z for both
186     PPO (test 1 = PPO pool) and greedy (test 1 = greedy pool).
187     """
188     env = DaptsBucketEnv(instance_sampler, B, G, N)
189     ppo_rewards, greedy_rewards = [], []
190     for ep in range(n_episodes):
191         obs, _ = env.reset(seed=seed + ep)
192         done = False
193         reward = 0.0
194         while not done:
195             action, _ = model.predict(obs, deterministic=True)
196             obs, reward, done, _, _ = env.step(action)
197             ppo_rewards.append(reward)
198             greedy_rewards.append(
199                 rollout_utility(list(env.p), list(env.u), B, G, env.last_z))
200     return float(np.mean(ppo_rewards)), float(np.mean(greedy_rewards))
201
202
203 # -----
204 # CLI
205 # -----
206
207 def _run_exact(args):
208     if args.source == "csv":
209         instances, N, B, G = load_csv_instances(args.csv)
210         p, u = instances[args.csv_row]
211         n = N

```

```

212     print(f"[exact] CSV {os.path.basename(args.csv)} row {args.csv_row}: "
213           f"n={n}, B={B}, G={G}")
214 else:
215     n, B, G = args.n, args.B, args.G
216     rng = np.random.default_rng(args.seed)
217     p, u = random_instance_sampler(n)(rng)
218     print(f"[exact] random instance (seed={args.seed}): n={n}, B={B}, G={G}")
219
220     print(f" p = {[round(x, 4) for x in p]}")
221     print(f" u = {[round(x, 2) for x in u]}")
222
223     env = DaptsExactEnv(fixed_instance_sampler(p, u), B, G, n)
224     model_path = os.path.join(args.model_dir, f"exact_n{n}_B{B}_G{G}_s{args.seed}")
225     print(f" training PPO for {args.timesteps} timesteps (seed={args.seed})...")
226     model = train(env, args.timesteps, args.seed, model_path,
227                  verbose=1 if args.verbose else 0)
228
229     ppo_val, dp_val = evaluate_exact_vs_dp(model, p, u, B, G)
230     ratio = ppo_val / dp_val if dp_val else float("nan")
231     print("\n--- Validación: PPO vs DP óptimo ---")
232     print(f" PPO (valor exacto de la política) : {ppo_val:.6f}")
233     print(f" DP óptimo : {dp_val:.6f}")
234     print(f" cociente PPO / DP : {ratio:.4f}")
235
236
237 def _run_bucket(args):
238     if args.source == "csv":
239         instances, N, B, G = load_csv_instances(args.csv)
240         sampler = csv_instance_sampler(instances)
241         print(f"[bucket] CSV {os.path.basename(args.csv)}: "
242               f"{len(instances)} instances, N={N}, B={B}, G={G}")
243     else:
244         N, B, G = args.N, args.B, args.G
245         sampler = random_instance_sampler(N)
246         print(f"[bucket] random instances (seed={args.seed}): "
247               f"N={N}, B={B}, G={G}")
248
249     env = DaptsBucketEnv(sampler, B, G, N)
250     model_path = os.path.join(args.model_dir, f"bucket_N{N}_B{B}_G{G}_s{args.seed}")
251     print(f" training PPO for {args.timesteps} timesteps (seed={args.seed})...")
252     model = train(env, args.timesteps, args.seed, model_path,
253                  verbose=1 if args.verbose else 0)
254
255     ppo_mean, greedy_mean = evaluate_bucket(
256         model, sampler, B, G, N, args.eval_episodes, args.seed + 10_000)
257     print(f"\n--- Evaluación sobre {args.eval_episodes} episodios ---")
258     print(f" PPO (utilidad media) : {ppo_mean:.4f}")
259     print(f" greedy (utilidad media) : {greedy_mean:.4f}")
260     print(f" diferencia PPO - greedy : {ppo_mean - greedy_mean:+.4f}")
261
262
263 def main(argv=None):
264     parser = argparse.ArgumentParser(description=__doc__)
265     sub = parser.add_subparsers(dest="env", required=True)
266
267     common = argparse.ArgumentParser(add_help=False)
268     common.add_argument("--source", choices=["random", "csv"], default="random")
269     common.add_argument("--csv", help="path to a classical data CSV")
270     common.add_argument("--B", type=int, default=2)
271     common.add_argument("--G", type=int, default=3)
272     common.add_argument("--timesteps", type=int, default=60_000)
273     common.add_argument("--seed", type=int, default=0)
274     common.add_argument("--model-dir", default=DEFAULT_MODEL_DIR)
275     common.add_argument("--verbose", action="store_true")
276
277     pe = sub.add_parser("exact", parents=[common],
278                        help="exact belief-MDP, validated against DP")
279     pe.add_argument("--n", type=int, default=6)
280     pe.add_argument("--csv-row", type=int, default=0)
281
282     pb = sub.add_parser("bucket", parents=[common],
283                        help="bucketed env for large N")

```

```
284 pb.add_argument("--N", type=int, default=50)
285 pb.add_argument("--eval-episodes", type=int, default=200)
286
287 args = parser.parse_args(argv)
288 if args.source == "csv" and not args.csv:
289     parser.error("--source csv requires --csv PATH")
290
291 if args.env == "exact":
292     _run_exact(args)
293 else:
294     _run_bucket(args)
295
296
297 if __name__ == "__main__":
298     main()
```